COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	Sep 19, 2026	**DATE SUBMITTED:**	Sep 25, 2026
NAME:	Formilleza, Huone Anthony	**INSTRUCTOR:**	Eng. Jimlord Quejado

ACTIVITY NO. 8.1 **[Hands-on Activity 8.1 Sorting Algorithms Pt2]**

1. PROCEDURE OUTPUT

Table 8-1. Array of Values for Sort Algorithm Testing

Item	Content / Observations

Code + Console Screenshot	`int originalArray[SIZE];`
	`int workingArray[SIZE];`
	
	`srand(time(0));`
	
	`for (int i = 0; i < SIZE; i++) {`
	`  originalArray[i] = rand() % 500 + 1;`
	`}`
	
	`cout << "Original unsorted array of 100 elements" << endl;`
	`printArray(originalArray, SIZE);`
	`cout << endl;`

Observations	The system basically threw together a random, messy batch of 100 numbers anywhere between 1 and 500. It didn't bother organizing them at all upfront, mostly just so there's a fair, chaotic starting point to see how the different algorithms handle it.

Table 8-2. Shell Sort Technique

Item	Content / Observations

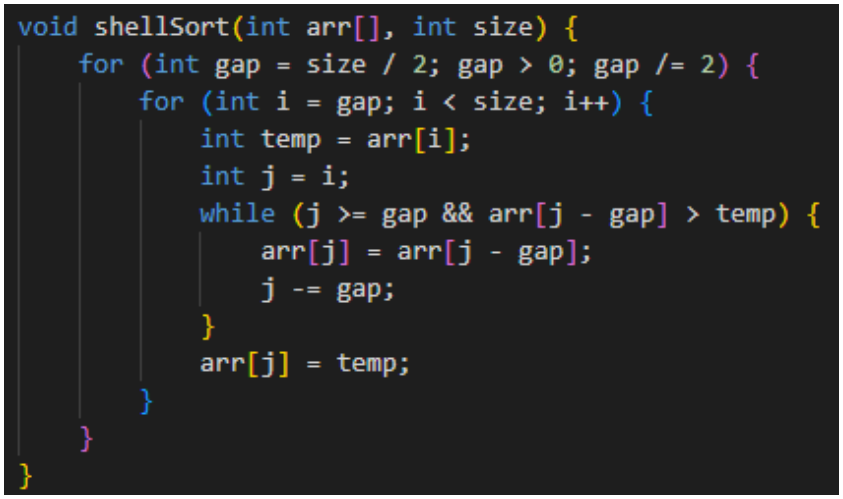
Code + Console Screenshot	
Observations	This one eventually managed to line everything up from smallest to largest. Instead of jumping straight to adjacent numbers, it takes a bit of a detour by comparing items sitting pretty far apart, slowly closing that gap pass by pass until it's just doing a quick final polish on a nearly organized list.

Table 8-3. Merge Sort Algorithm

Item	Content / Observations

Code + Console Screenshot

```cpp
void merge(int arr[], int left, int mid, int right) {
    vector<int> temp;

    int i = left;
    int j = mid + 1;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp.push_back(arr[i]);
            i++;
        } else {
            temp.push_back(arr[j]);
            j++;
        }
    }

    while (i <= mid) {
        temp.push_back(arr[i]);
        i++;
    }

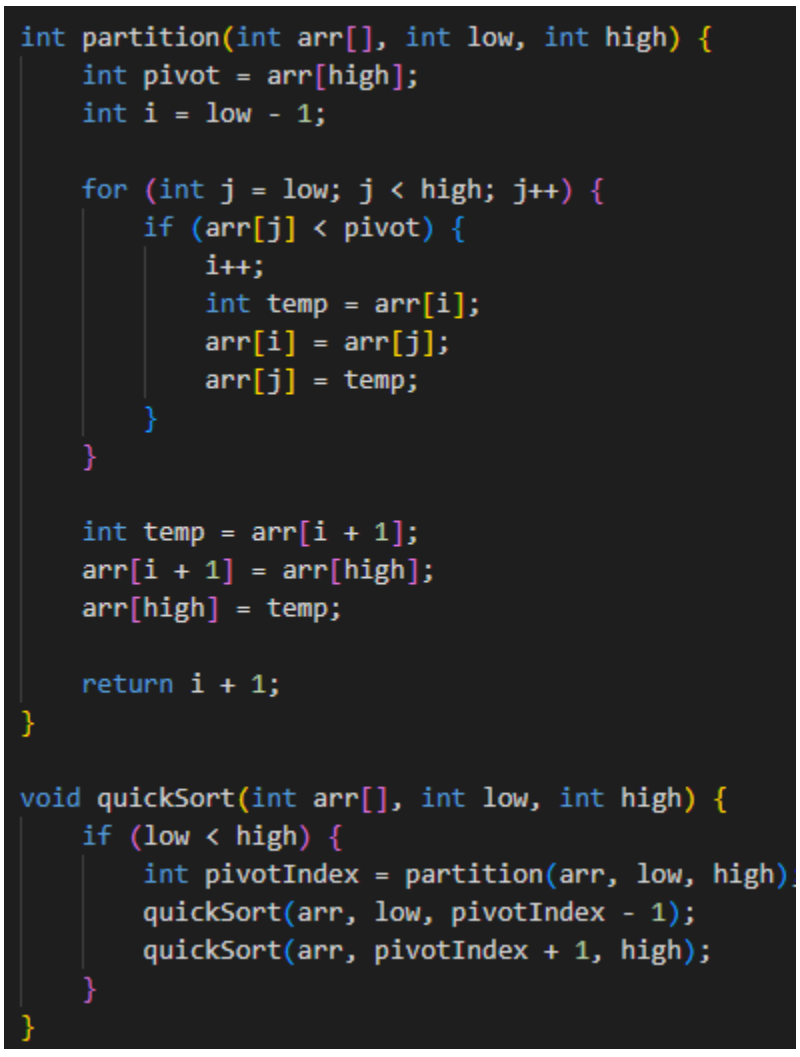
    while (j <= right) {
        temp.push_back(arr[j]);
        j++;
    }

    for (int k = 0; k < temp.size(); k++) {
        arr[left + k] = temp[k];
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

Observations	Got all 100 numbers sorted without a hitch. Its strategy is a bit round-about—it keeps breaking the list down into smaller and smaller halves until it's left with individual numbers, and only then does it start carefully stitching them back together in proper order.

Table 8-4. Quick Sort Algorithm

Item	Content / Observations
Code + Console Screenshot	

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivotIndex = partition(arr, low, high)
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

Observations	Wrapped up the sorting job successfully, too. It singles out the very last number to act as a anchor, shuffles smaller stuff to the left and bigger stuff to the right, and then keeps repeating that same dance on each section until everything happens to land where it belongs.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other
sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

One of the two algorithms would be quick sort since a randomly shuffled set of numbers is likely to be randomized enough for the method to perform at its peak efficiency. The merge sort algorithm is also a good candidate because its running time is consistent, regardless of the incoming data sets' organization. Both algorithms have the average efficiency of O(n log n) because they split the array or list of numbers into halves that require log n operations to reach the bottom of the recursion and process every element, which amounts to O(n) operations overall. The only significant problem with quick sort is the choice of pivots, which, if not done properly, can make the algorithm run at the worst-case $O(n^2)$, when the data sets are already sorted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Fixing my grammar, and asked for suggestion on the code

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used gemini to suggest fixing some code errors and explain them to me**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.